

# Responses to Trivial Relocation NB Comments

Document #: P3915R0  
Date: 2025-11-07 10:48 HST  
Project: Programming Language C++  
Audience: EWG  
Reply-to: Pablo Halpern  
<[phalpern@halpernwrightsoftware.com](mailto:phalpern@halpernwrightsoftware.com)>

## Contents

- 1 Introduction
- 2 What Trivial Relocation is
- 3 Categories of Trivial Relocation NB Comments
- 4 Category 1: Remove Just about Everything From P2686
- 5 Category 2: Make Trivial Relocation the Same as memcpy
  - 5.1 NB Comments Addressed
  - 5.2 Assertions made in the NB Comments
  - 5.3 The Reality of Treating Objects as Bytes
  - 5.4 Aren't Unions just as Problematic?
  - 5.5 What Can We Do for Byte Buffers?
  - 5.6 Options for Proceeding
  - 5.7 Option 1: Status Quo
  - 5.8 Option 2: Require that memcpy perform trivial relocation
  - 5.9 Option 3: Require that `trivially_relocate` be a bitwise operation
  - 5.10 Option 4: Make most unions not TR
  - 5.11 Option 5: Make polymorphic types not TR (or make it implementation-defined)
  - 5.12 Conclusion: Goals to Guide our Decision
  - 5.13 Postscript: A trait for Detecting TR Within a Union
- 6 Category 3: Replace `std::relocate` with `std::uninitialized_relocate_*`

- 6.1 NB Comments Addressed
- 6.2 Commonalities between `std::relocate` and `std::uninitialize_relocate_*`
- 6.3 Differences between `std::relocate` and `std::uninitialize_relocate_*`
- 6.4 A sample implementation of `vector<T>::erase`
- 6.5 Does Throwing `uninitialized_relocate` Result in Cleaner Code?
- 6.6 Raw Pointers vs. Iterators
- 6.7 Options that Might Increase Consensus
- 7 Category 4: Fix Implicit Trivial Relocatability
  - 7.1 NB Comment Addressed
  - 7.2 What's Broken #1
  - 7.3 What's Broken #2
  - 7.4 The Source of the problem
  - 7.5 Proposed Resolution
- 8 Category 5: Conditional Trivial Relocation
  - 8.1 NB Comments Addressed
  - 8.2 Assertions made in the NB Comments
  - 8.3 Observations
- 9 Category 6: Allow `const` Qualified Types for Relocation Functions
  - 9.1 NB Comments Addressed
  - 9.2 Observations
- 10 Category 7: P3858: `restart_lifetime`
- 11 References

## § 1 Introduction

- This is the [P2786R13] authors' response to NB comments about Trivial Relocation.
- NB comments are grouped into similar concerns.
- Where appropriate, options are provided.
- Issues about which the P2786 authors do not have a strong feeling have neutral observations.

## § 2 What Trivial Relocation is

Moving an object *of known type* from one location to another

- without invoking any constructors or destructors,
- ending the lifetime of the object at the source location, and
- starting the lifetime of the object at the target location

The trivial relocation operation is specified in terms of the abstract machine, object lifetimes, and object representation, without making any other implementation assumptions.

## § 3 Categories of Trivial Relocation NB Comments

1. **US 10-023**: Remove Just about Everything From P2686
2. **CN 1, US 46-085, US 45-081, BDS2-034**: Make Trivial Relocation the Same as memcpy
3. **US 73-131, US 72-130, US 166-266, CA-133**: Replace `std::relocate` with `std::uninitialized_relocate_*`
4. **US 47-084**: Fix Implicit Trivial Relocatability
5. **FR-002-025, US 44-082**: Conditional Trivial Relocation
6. **CA-136, CA-137**: Allow `const` Qualified Types for Relocation Functions
7. **RO 1-134, US 74-135: P3858**: `restart_lifetime`

## § 4 Category 1: Remove Just about Everything From P2686

NB Comment Addressed: **US 10-023**: Remove `*_if_eligible` keywords

- This NB comment is a summary of the arguments made in P1144R13.
- All of the arguments have been discussed multiple times, most recently in Sofia in June 2025.
- There was consensus against using committee time to explore further the ideas in P1144:

|             | SF | F | N | A  | SA |
|-------------|----|---|---|----|----|
| <b>EWG</b>  | 8  | 6 | 4 | 14 | 20 |
| <b>LEWG</b> | 1  | 2 | 2 | 7  | 11 |

- US 10-023 adds no new information to substantiate the claims that failed to persuade in Sofia and prior.

## § 5 Category 2: Make Trivial Relocation the Same as memcpy

### § 5.1 NB Comments Addressed

- **BDS2-034**: Permit trivial relocation via memcpy
- **CN 1**: Trivial relocatability should imply bitwise relocatability
- **US 45-081**: Satisfy expectations that trivially relocatable types are memcpy-able
- **US 46-085**: Make trivial relocation implementable by memcpy

### § 5.2 Assertions made in the NB Comments

If we could memcpy, memmove, and realloc objects of types for which `is_trivially_relocatable_v<T>` is true, then we could:

1. Allow existing (mis-)uses of memcpy to become valid and
2. Enable trivial relocation of byte-buffers containing (type-erased) nested objects.

### § 5.3 The Reality of Treating Objects as Bytes

- A crucial part of the C++ object model is that, except for a narrow set of *implicit lifetime* types (those having a trivial constructor and a trivial destructor),
  - an object's lifetime starts when its constructor completes or on return from `start_lifetime_as` or `trivially_relocate`; and
  - an object's lifetime ends when its destructor starts, its storage is reused, or on entry to `trivially_relocate` (see 6.8.4 [\[basic.life\]](#)<sup>1</sup>)
- An implementation cannot start or end the lifetime of an object within an untyped bag of bytes; memcpy loses critical type information with no means of recovery.
- EWG and CWG have repeatedly rejected attempts to relax the object model. Specifically they objected to bypassing constructors in an early trivial-relocation paper, [\[N4034\]](#), way back in **2014**

### § 5.4 Aren't Unions just as Problematic?

- Union members have well-defined type semantics in the abstract machine; they are not untyped bags of bytes.
- The current TR wording describes what is known to be implementable, including by sanitizers that track the active member.
  - For all known architectures, the compiler can reconstruct the object representation of the active TR member.

- There is an implementation-defined (compile-time detectable) carve out for polymorphic types within unions.
- Future architectures might expand the set of types that might need a special TR carve out within unions.

## § 5.5 What Can We Do for Byte Buffers?

- The current TR spec does not work for byte buffers providing storage for nested objects.
- Any attempt to make it “just work” would weaken the object model and might not be implementable, especially if specified in a hurry.
- However, a soon-to-be-proposed *general* relocation facility will involve *relocation constructors* that can perform TR on such nested objects.
- In the meantime, anything that worked before will continue to work, if you know your architecture and compiler.

## § 5.6 Options for Proceeding

1. Status quo (no change to the CD)
2. Require that trivial relocation be the same as memcpy
3. Require that `trivially_relocate` be a bitwise operation
4. Make most unions not TR
5. Make polymorphic types not TR (or make it implementation-defined)

## § 5.7 Option 1: Status Quo

**The CD is not broken;** it just has some limitations.

- **Does not allow TR of objects nested within byte buffers**
- It is sufficient for most potentially trivially relocatable types, including `std::optional` and `std::variant` holding TR types.
- Performance benefit for many, many types;
  - Usually implemented with memmove-like efficiency
  - Concessions for polymorphic types on new architectures like ARM64e do not penalize the vast majority of types (even on ARM64e) or implementations.
- A future generalized relocation feature would allow the user to recover lost type information for relocating of objects nested in byte buffers.

## § 5.8 Option 2: Require that memcpy perform trivial relocation

- **Would allow TR of objects with type-erased byte buffers**
- Not implementable
  - Loss of type information for compiler and sanitizers
  - Brittle in the face of evolving hardware architectures and higher-quality optimizers.
- Radically weakens the object model lifetime guarantees
  - Begins and ends lifetimes without any indication that it's doing so
  - It would be disastrous for existing code if `memcpy` ended the lifetime of objects nested within the source buffer.

## § 5.9 Option 3: Require that `trivially_relocate` be a bitwise operation

- ***Might* allow TR of objects nested within byte buffers** (though rejected by previous CWG review).
- Hard to specify *bitwise* within the object model
  - previously rejected by CWG.
  - Might be impossible to reconcile with `constexpr` evaluation.
  - Too late for C++26
- Although some think that *trivial* implies bitwise, we believe a more useful formulation is that *trivial* implies no user code is invoked.
- Reduces the set of eligible TR types on certain architectures (with more such architectures to come)
- Desired functionality could wait until C++29 if we later add a new trait, e.g., `is_bitwise_trivially_relocatable`, per [P3780R0]

## § 5.10 Option 4: Make most unions not TR

- **Does nothing to allow TR of objects nested within byte buffers**
- Only unions of *implicit-lifetime* TR types would themselves be TR.
- Requires less compiler magic on exotic architecture than does the status quo.
- No longer possible to create TR `optional` or `variant` other than for trivially copyable types

## § 5.11 Option 5: Make polymorphic types not TR (or make it implementation-defined)

- **Does nothing to allow TR of objects nested within byte buffers**

- Simplifies specification slightly by eliminating the distinction between *trivially relocatable* and *trivially relocatable within a union*.
- Excludes useful a set of types from TR.
- Gives us the opportunity to do something better in a future standard

## § 5.12 Conclusion: Goals to Guide our Decision

- Allow the widest reasonable set of types to be TR
- Make sure object lifetimes are managed
- Make it possible to create a TR optional and variant holding TR elements.
- Do not penalize all platforms for the restrictions of some (currently minority) cases
- Do not compromise the object model for a handful of use cases, many of which can be accommodated with the right implementation strategy
- Leave room for future improvements (e.g., relocation constructors).

## § 5.13 Postscript: A trait for Detecting TR Within a Union

```
template <class T>
union __union_of { T x; __union_of() {} ~__union_of() {} };

template <class T>
struct is_trivially_relocatable_in_union :
    is_trivially_relocatable<__union_of<T>> { };
```

# § 6 Category 3: Replace `std::relocate` with `std::uninitialized_relocate*`

## § 6.1 NB Comments Addressed

- CA-133: Remove `std::relocate`
- US 72-130: adopt `std::uninitialized_relocate`
- US 73-131: Cleaning up the trivial relocation APIs
- US 166-266: P3516 Adopt Uninitialized algorithms for relocation

## § 6.2 Commonalities between `std::relocate` and `std::uninitialize_relocate*`

- `constexpr` function templates
- Use trivial relocation when possible
- Fall back to move-construct + destroy if trivial relocation is not available

- Can be entirely written as library functions on top of `is_trivially_relocatable`, `trivially_relocate`, `is_move_constructible`, etc.

## § 6.3 Differences between `std::relocate` and `std::uninitialized_relocate_*`

| <code>std::relocate</code> (CD and P2786)                      | <code>std::uninitialized_relocate_*</code> (P3516)                                                                                              |
|----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Does not throw (requires <code>is_nothrow_relocatable</code> ) | Might throw                                                                                                                                     |
| Works with raw pointers                                        | Works with iterators                                                                                                                            |
| Automatically detects direction                                | separate forward & reverse functions                                                                                                            |
| Single function template                                       | 16 function templates and overloads including parallel and range versions consistent with other <code>uninitialized_*</code> function templates |

Future proposals for relocation will involve *relocation constructors*, which cannot throw, lest both the source and destination be left in an indeterminate, non-destructible state.

## § 6.4 A sample implementation of `vector<T>::erase`

Below, we have two implementations of `vector<T>::erase` (with allocator details elided for brevity). The implementation on the left works with either `std::relocate` from the CD and `std::uninitialized_relocate` from [P3516R2]. The implementation on the right works only with `std::uninitialized_relocate`, taking advantage of its built-in cleanup if an invoked move constructor throws an exception. The purpose of this example is to explore whether allowing throwing within `std::uninitialized_relocate` provides tangible benefits in code complexity and/or efficiency.

| Using Nothrow Relocation                                                                                                                                                                                                                                                                                                                                                                                                         | Using Throwing <code>std::uninitialized_relocate</code>                                                                                                                                                                                                                                                                                                                                                       |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> iterator erase(const_iterator pos) {     auto idx = pos - cbegin();     T* p = __start + idx; // mutable                         ptr     if constexpr         (is_nothrow_relocatable_v&lt;T&gt;)     {         p-&gt;~T();         [uninitialized_]relocate(p + 1,                                 __last, p);     }     else {         for (++p ; p &lt; __last; ++p)             *(p-1) = std::move(*p);     } } </pre> | <pre> iterator erase(const_iterator pos) {     auto idx = pos - cbegin();     iterator mpos = begin() + idx; //                                 mutable iter     mpos-&gt;~T();     try {         uninitialized_relocate(mpos +                                1, cend(), mpos);         --__last;         return mpos;     }     catch (...) {         __last = __start + idx;         throw;     } } </pre> |



| Using Nothrow Relocation                                     | Using Throwing<br><code>std::uninitialized_relocate</code> |
|--------------------------------------------------------------|------------------------------------------------------------|
| <pre> p-&gt;~T(); } --__last; return begin() + idx; } </pre> | <pre> } } </pre>                                           |

## § 6.5 Does Throwing `uninitialized_relocate` Result in Cleaner Code?

- When inserting or erasing into a vector-like or deque-like container:
  - A call to a throwing `uninitialized_relocate` can avoid a separate assignment fall-back loop **but**
  - it requires `try/catch` to fix up the container upon failure.
- A throwing move constructor is almost always correlated with a throwing (allocating) default constructor.
  - The move constructor allocates and the destructor deallocates.
  - Such types often have non-allocating (and thus non-throwing) move-assignment operators (e.g., MSVC `std::list`)
  - Such types can often be trivially relocated, but not until TR has been in the standard for some time.
  - Therefore, move-delete is almost always a pessimization for types having a throwing move constructor.
- Our conclusion is that allowing a `uninitialized_relocate` to be a throwing operation provides marginal benefit, at best, and might encourage inefficient code.

## § 6.6 Raw Pointers vs. Iterators

- Raw pointers are simple, but very low-level
  - They are the lingua-franca of low-level operations such as placement `new` and `begin_lifetime_as`.
- Iterators are marginally more type-safe, but it is still easy to provide iterators from different containers, etc.
- Ending the lifetime of an object through an iterator puts the source container in a radioactive state.
  - `std::destroy` is not a good model
- A pointer *can* convey ownership, whereas an iterator *never* does.
- Ultimately it is somewhat an aesthetic choice, though at least one P2786 author is very unhappy with the [P3516R2] iterator interface.

## § 6.7 Options that Might Increase Consensus

- Remove `std::relocate` from the CD, but do not replace it with anything. Allow library vendors to organically discover the best interface for implementing `std::vector`, `std::deque`, etc.
- Replace `relocate` with *non-throwing* `uninitialized_relocate_*`.
- Rename `uninitialized_relocate_*` to `uninitialized_move_destroy_*` or `uninitialized_destructive_move_*`.
  - Keep `std::relocate` as is
  - Preserve the term, *relocate*, to refer to a no-fail operation (including for future user-defined relocation operations)
- Status quo: Move forward with `uninitialized_relocate_*`, but keep `relocate`
  - `uninitialized_relocate_*` could be added to C++26 or C++29
- Although the authors of P2786 oppose replacing `std::relocate` with `uninitialized_relocate` as defined in [P3516R2], this option is a reasonable way to make the TR feature usable.

## § 7 Category 4: Fix Implicit Trivial Relocatability

### § 7.1 NB Comment Addressed

- US 47-084 (referencing issue [CWG3049]): Implicitly Deleted Move Operation Should not Disable Trivial Relocation

### § 7.2 What's Broken #1

The design exposition for P2786 states:

- Any trivially copyable type is trivially relocatable by default.

However, the following types are not trivially relocatable according to the CD, despite being trivially copyable and not opting out in any way:

```
struct A { const int i; }; // deleted assignment; not TR according to the
CD
struct B { int&      r; }; // deleted assignment: not TR according to the
CD
```

### § 7.3 What's Broken #2

The design exposition for P2786 states:

- Examples of types that are **implicitly trivially relocatable** are trivially copyable types (such as scalar types); aggregates of *trivially relocatable types*, including

arrays of such types; and such aggregates with `const` and/or reference data members.

However, type D below is not trivially relocatable despite being a simple aggregate of trivially relocatable types.

```
struct C trivially_relocatable_if_eligible { // trivially relocatable
    C(){}
    C(const C&) = delete;
};
struct D { C m; }; // deleted move constructor; not TR according to the
                  CD
```

## § 7.4 The Source of the problem

11.2 [class.prop]/2:

2 A class C is *default-movable* if

- (2.1) — overload resolution for direct-initializing an object of type C from an xvalue of type C selects a constructor that is a direct member of C and is neither user-provided **nor deleted**. [Emphasis mine]
- (2.2) — overload resolution for assigning to an lvalue of type C from an xvalue of type C selects an assignment operator function that is a direct member of C and is neither user-provided **nor deleted**. [Emphasis mine]
- (2.3) — C has a destructor that is neither user-provided **nor deleted**. [Emphasis mine]

The authors of P2786 recognize this issue as a cut-and-paste error introduced when refactoring the wording section.

## § 7.5 Proposed Resolution

The authors of P2786 are in favor the following resolution:

Change 11.2 [class.prop]/2 as follows.

2 A class C is *default-movable* if

- (2.1) — overload resolution for direct-initializing an object of type C from an xvalue of type C selects a constructor that is a direct member of C and is neither user-provided nor explicitly deleted.
- (2.2) — overload resolution for assigning to an lvalue of type C from an xvalue of type C selects an assignment operator function that is a direct member of C and is neither user-provided nor explicitly deleted.
- (2.3) — C has a destructor that is neither user-provided nor explicitly deleted.

## § 8 Category 5: Conditional Trivial Relocation

### § 8.1 NB Comments Addressed

- FR-002-025: Make `*_if_eligible` real keywords
- US 44-082: Conditional trivially relocatable types

### § 8.2 Assertions made in the NB Comments

- To make a class trivially relocatable based on a compile-time property (e.g., whether a nested object type is TR) requires cumbersome metaprogramming.
- Adding a language-supported predicate to the TR keywords in the future would be difficult because of a parsing ambiguity:

```
template <class T>
struct X trivially_relocatable_if_eligible(some_trait<T>) ...
```

Is the above a declaration of X or the definition and initialization of `trivially_relocatable_if_eligible` of type `struct X`, initialized with `some_trait<T>`?

### § 8.3 Observations

- In almost all circumstances, the implicit eligibility requirements provide conditional TR without needing a new syntax or work-around.
- The most common exception is for nested objects of template-parameter type stored in untyped byte buffers (e.g., `variant` implemented using a byte buffers rather than a `union`)
  - The byte buffers would allow the instantiated class to be TR **but**
  - the instantiation should not be TR unless the objects nested within the byte buffers are of *trivially copyable* types.
  - Hence, such a class template needs some conditional TR mechanism
  - The mechanism can be encapsulated in a (not-necessarily-standard) class template:

```
template <class... T> struct
__nested_obj_buffer;
```

Where a `__nested_obj_buffer` contains a byte buffer of sufficient size and alignment for one object of any type in `T...`

This class will be trivially relocatable iff all types in `T...` can be bitwise relocated (i.e., they are all *trivially copyable*).

- In almost all cases, a `union` would be a better choice, and would automatically produce the correct eligibility.
- Previous versions of P2786 supported conditional TR and had wording (with CWG input) that resolved the parsing ambiguity without using real keywords.
- Either or both changes (making the keywords non-contextual or adding a boolean condition to the keywords) could be applied in C++29 without any more chance of breakage than today.

The authors of P2786 are mostly neutral on whether to make `*_if_eligible` real (non-contextual) keywords or whether they should have a conditional predicate, despite seeing little need for either. It is too late in the process to consider any of the redesign alternatives described in US 44-082, and we believe that the current mitigation strategies are sufficient to avoid the need to remove or dramatically alter the entire trivial relocation feature.

## § 9 Category 6: Allow `const` Qualified Types for Relocation Functions

### § 9.1 NB Comments Addressed

- CA-136: Allow `const`-qualified types for `relocate`
- CA-137: Allow `const`-qualified types for `trivially_relocate`

Both comments would allow `const`-qualified `T` for the `T*` template arguments, claiming consistency with constructors and destructors, which do work on `const`-qualified types.

### § 9.2 Observations

- Relocation is conceptually different from construction and destruction, although non-trivial relocation does invoke constructors and destructors.
- *Currently* relocation is rarely possible on objects with non-dynamic lifetimes, so there are few use cases where a `const`-qualified argument would come into play.
- As usual, `const`-qualification is a safety feature that can be worked around with `const_cast` in the rare case where it's problematic.

The authors of P2786 are mostly neutral on the proposed changes, despite not seeing a strong need for them. The two changes should be consistent — if one is adopted, the other should also be adopted.

## § 10 Category 7: P3858: `restart_lifetime`

NB Comments Addressed

- RO 1-134: Add a `start_lifetime_at` [sic] function
- US 74-135: Add a `start_lifetime_at` [sic] function P3858

Although one of the authors of P2786 is also an author of P3858, we feel that the proposed feature is too novel and untested for C++26. The facility described in [P3858R0] are not needed for trivial relocation to be a complete and useful feature, nor is there anything in the CD that would prevent adoption of such a facility in the future.

## § 11 References

[CWG3049] Pablo Halpern. 2025-07-10. Implicitly deleted move operation should not disable trivial relocation.

<https://wg21.link/cwg3049>

[N4034] Pablo Halpern. 2014-05-27. Destructive Move.

<https://wg21.link/n4034>

[P2786R13] Pablo Halpern, Joshua Berne, Corentin Jabot, Pablo Halpern, Lori Hughes. 2025-02-14. Trivial Relocatability For C++26.

<https://wg21.link/p2786r13>

[P3516R2] Louis Dionne, Giuseppe D'Angelo. 2025-05-16. Uninitialized algorithms for relocation.

<https://wg21.link/p3516r2>

[P3780R0] Giuseppe D'Angelo. 2025-07-10. Detecting bitwise trivially relocatable types.

<https://wg21.link/p3780r0>

[P3858R0] David Sankel, Jon Bauman, Pablo Halpern. 2025-10-06. A Lifetime-Management Primitive for Trivially Relocatable Types.

<https://wg21.link/p3858r0>

- 
1. All citations to the Standard are to working draft N5014 unless otherwise specified. ↩